

Ledgerline: AI-Powered Credit Card Approval Prediction

Project Description

Ledgerline automates the credit card approval decision using machine learning. The platform includes a single-applicant prediction module, which returns an instant Approve/Decline decision with a calibrated confidence score; a Batch Screening module, which scores an entire CSV of applicants at once for compliance review; and a Model Transparency Report, which shows how four trained classifiers compare so the decision process is auditable rather than a black box.

Built with Flask and four scikit-learn/XGBoost classifiers trained on applicant financial and credit-history data, Ledgerline processes income, employment, demographic, and payment-history inputs to deliver fast, consistent credit decisions. With a calibrated decision threshold (rather than a raw 0.5 cutoff) and a clear separation between data, model, and application code, Ledgerline empowers credit analysts, compliance officers, and prospective customers to get consistent, explainable answers in under a second.

Scenarios

Scenario 1 — Automated Credit Card Application Screening: A bank credit analyst enters a new applicant's financial profile — income type, employment duration, credit history — into the web application. The model returns an instant approval or rejection prediction with a confidence score, letting the analyst prioritize applications that need further manual review.

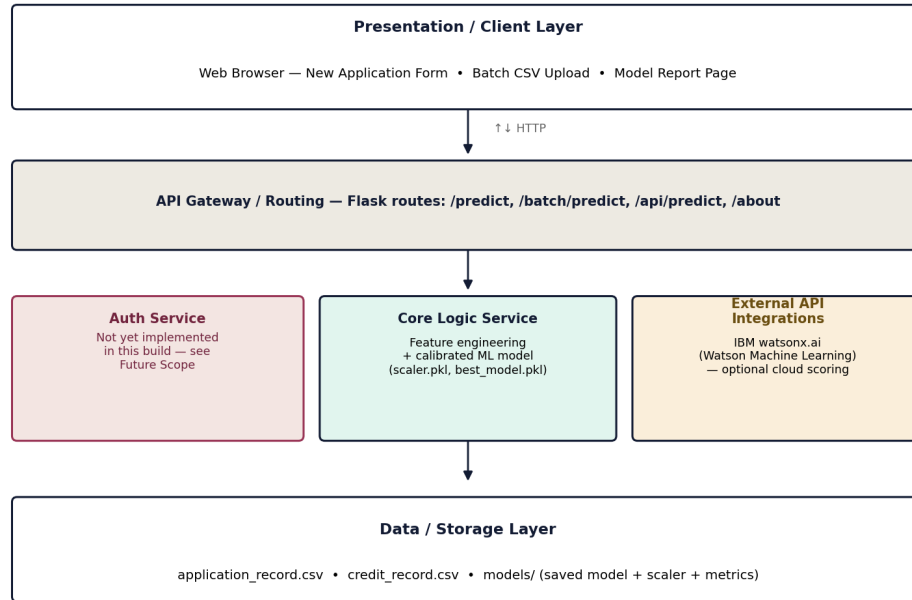
Scenario 2 — High-Risk Applicant Identification and Compliance Review: A compliance officer uploads a CSV of applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into a binary risk label, so the model clearly classifies high-risk applicants as ineligible — across the whole batch in one pass.

Scenario 3 — Customer Self-Service Credit Card Eligibility Check: A prospective customer enters personal and financial details before submitting a formal application. The system instantly predicts approval likelihood, helping the customer understand their eligibility and reducing unnecessary applications (and the credit inquiries that come with them).

Technical Architecture

Ledgerline uses a layered architecture: a browser-based Presentation Layer (three pages — new application, batch screening, model report), a Flask routing layer acting as the API gateway, a Core Logic Service that engineers features and scores them with the calibrated model, and a Data/Storage layer (CSV source data plus serialized model artifacts). An optional External API Integration layer connects to IBM watsonx.ai for cloud-hosted scoring.

Solution Architecture — Credit Card Approval Prediction



(Note: this project uses flat CSV files rather than a database, so no ER diagram is included — `application_record.csv` and `credit_record.csv` are merged and feature-engineered directly into the model-ready dataset.)

Pre-requisites

- Python Programming Proficiency: docs.python.org
- Flask Framework Knowledge: flask.palletsprojects.com
- scikit-learn & XGBoost Familiarity: scikit-learn.org, xgboost.readthedocs.io
- HTML & CSS Skills: developer.mozilla.org
- Version Control with Git: git-scm.com/doc
- Jupyter Notebook for data analysis: jupyter.org
- (Optional) IBM watsonx.ai for cloud deployment: ibm.com/products/watsonx-ai

Project Workflow

Milestone 1: Data Preparation and Model Selection

- Activity 1.1: Generate a synthetic dataset matching the schema of the public “Credit Card Approval Prediction” dataset (`application_record.csv` + `credit_record.csv`), since the build environment had no internet access to Kaggle.
- Activity 1.2: Make risk genuinely depend on income, employment length, age, and dependents (not pure noise), so the trained models would have real signal to learn from.
- Activity 1.3: Research and select the four required algorithms — Logistic Regression, Decision Tree, Random Forest, XGBoost — matching the project brief’s required skill set.
- Activity 1.4: Set up the development environment, installing Flask, scikit-learn, XGBoost, pandas, NumPy, and matplotlib.

Milestone 2: Core Functionality Development

- Activity 2.1: Build the feature-engineering pipeline — merge application and credit records, convert multi-class STATUS codes into a binary risk label (60+ days past due = high risk), derive AGE_YEARS / YEARS_EMPLOYED / INCOME_PER_FAMILY_MEMBER, and one-hot encode categoricals.
- Activity 2.2: Train and compare all four classifiers on Accuracy, Precision, Recall, F1, and ROC-AUC; select the best by ROC-AUC.
- Activity 2.3: Discover during testing that class-balanced training pushed too many borderline-good applicants to ‘Decline’; fix this by removing class balancing and calibrating a probability threshold that maximizes F1 on held-out data instead of using a raw 0.5 cutoff.

Milestone 3: App.py Development

- Activity 3.1: Define Flask routes for each feature, each serving a distinct purpose:
/ — renders the new-application form
/predict — scores a single applicant and renders the result page
/api/predict — JSON API version of the same logic
/batch — renders the CSV upload page
/batch/predict — scores every row of an uploaded CSV and renders a results table
/about — renders the model comparison report

- Activity 3.2: Write `build_feature_row()`, a single shared function that converts raw form/CSV fields into the exact one-hot-encoded feature vector the model expects — used by every prediction route, so single and batch scoring can never drift apart.
- Activity 3.3: Write `predict_row()`, which scales the feature row and applies the calibrated decision threshold (loaded from `model_info.json`) rather than the model’s raw default.

Milestone 4: Frontend Development

- Activity 4.1: Design a custom “ledger” visual identity — a navy “spine” navigation column, paper-toned panels, and a rubber-stamp motif for the Approve/Decline decision — instead of a generic template look.

- Activity 4.2: Build four Jinja2 templates extending a shared base.html: the application form, the result page (with the stamp), the batch screening page (with a ledger-style results table), and the model report page.
- Activity 4.3: Add small, purposeful CSS entrance animations (the stamp “landing”, panels fading up, ledger rows revealing in sequence) that fully respect prefers-reduced-motion and leave the final appearance pixel-identical to the static design.

Milestone 5: Deployment

- Activity 5.1: Run and verify the application locally with the Flask development server.
- Activity 5.2: Write `deploy_watson.py` for optional cloud deployment to IBM watsonx.ai (Watson Machine Learning) — stores the model, creates an online deployment, and scores a test request. Written to IBM's current documented SDK pattern (`ibm-watsonx-ai`, since the older `ibm-watson-machine-learning` package is now in maintenance mode); not yet run against live IBM Cloud credentials.

Milestone 6: Conclusion

Ledgerline demonstrates a complete, working credit card approval prediction system: from raw applicant and credit-history data, through feature engineering and binary risk labeling, to a compared and calibrated machine learning model deployed in a usable, purpose-designed web application. All three target scenarios from the original brief — analyst screening, compliance batch review, and customer self-service — are functional and tested.

Exploring the Website's Web Pages

New Application Page

The single-page entry point for both credit analysts and self-service customers. A navy “spine” on the left provides navigation; the right-hand paper panel holds a sectioned form (Personal, Financial & Employment, Contact on File) ending in a “Run decision” button.

The screenshot shows the 'New credit card application' form on the Ledgerline Credit Decisioning website. The form is divided into three main sections: PERSONAL, FINANCIAL & EMPLOYMENT, and CONTACT ON FILE. The left sidebar contains navigation links: 'New application' (highlighted), 'Batch screening', and 'Model report'. The bottom of the sidebar mentions 'Model live engine: Logistic Regression'.

PERSONAL

Gender M	Age (years) 34
Family status Married	Family members (total) 2
Number of children 0	Education Secondary / secondary special

FINANCIAL & EMPLOYMENT

Annual income (\$) 55000	Income type Working
Years at current employer (0 if pensioner / not employed) 3	Occupation Laborers
Owns property Yes	Owns a car Yes
Housing situation House / apartment	

CONTACT ON FILE

☐ Work phone provided	☒ Personal phone provided
☒ Email provided	

Run decision →

Decision Result Page

After submitting, the applicant's decision is shown as a rotated rubber-stamp graphic (green ink for Approved, red for Declined), followed by a ledger-style breakdown of the approval confidence, decline risk, model engine used, and the key applicant details on file.

Ledgerline
CREDIT DECISIONING

New application

Batch screening

Model report

Model live
Engine: Logistic Regression

UNDERWRITING RESULT

Application decision

APPROVED

Approval confidence	73.9%
Decline risk	26.1%
Model engine	Logistic Regression
Annual income on file	\$95,000
Years employed	8
Income type	Working

New application

Batch Screening Page

Compliance officers upload a CSV here. Results appear as ledger-row totals (screened / approved / declined) followed by a per-applicant table with an inline decision stamp and decline-risk percentage for each row, plus a button to download the full results.

Ledgerline

CREDIT DECISIONING

New application

Batch screening

Model report

COMPLIANCE REVIEW

Batch applicant screening

Upload a CSV of applicants to screen them all at once. Each row is run through the same model used for single applications, so high-risk applicants are flagged consistently at any volume.

Choose File | No file chosen

CSV columns: gender, age, family_status, fam_members, children, education_type, income, income_type, years_employed, occupation_type, own_realty, own_car, housing_type, work_phone, phone, email

Run batch screen

844

SCREENEDAPPROVEDDECLINED

INCOME	INCOME TYPE	YEARS EMPLOYED	FAMILY STATUS	DECISION	DECLINE RISK
\$95,000	Working	8	Married	APPROVED	31.0%
\$250,000	Working	20	Married	APPROVED	0.3%
\$27,000	Student	0	Single / not married	DECLINED	93.6%
\$45,000	Working	2	Single / not married	DECLINED	64.0%
\$68,000	Pensioner	0	Widow	APPROVED	10.9%
\$38,000	Commercial associate	1	Civil marriage	DECLINED	85.5%
\$180,000	State servant	30	Married	APPROVED	0.7%
\$22,000	Student	0	Single / not married	DECLINED	91.5%

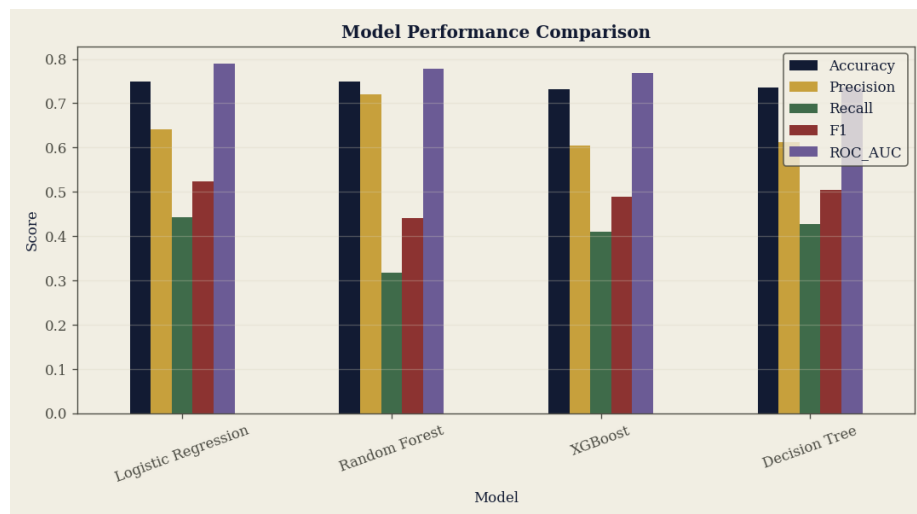
Download full results (CSV)

Model live

Engine: Logistic Regression

Model Report Page

Shows the head-to-head comparison of all four trained models — Accuracy, Precision, Recall, F1, and ROC-AUC — so reviewers can see exactly why Logistic Regression was selected as the production model in this build.



Conclusion

Ledgerline turns a slow, manual, inconsistent credit-approval process into an instant, consistent, and explainable one — without sacrificing transparency. By comparing four algorithms instead of committing to one, calibrating the decision threshold instead of trusting a default cutoff, and sharing one preprocessing path between single and batch scoring, the system stays both accurate and auditable. The clearest next step is retraining on the real

applicant dataset once available, and completing the live IBM Watson Machine Learning deployment for cloud-hosted scaling.